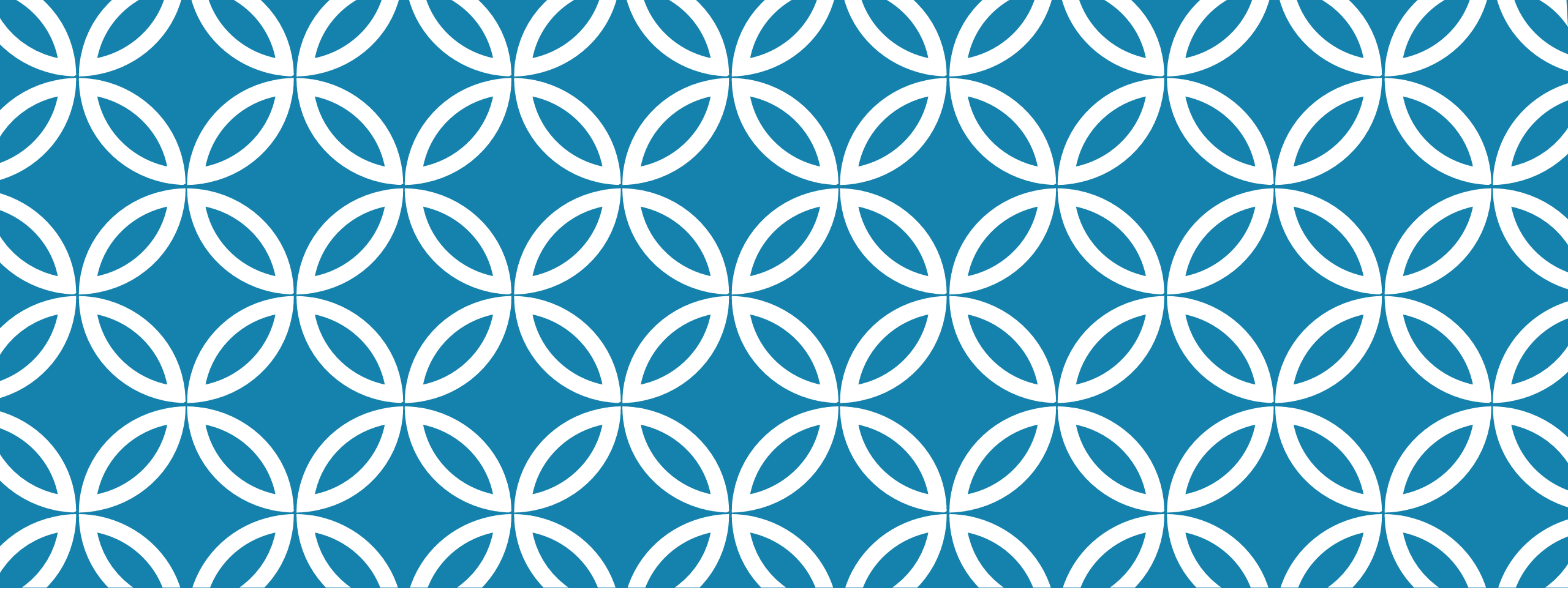
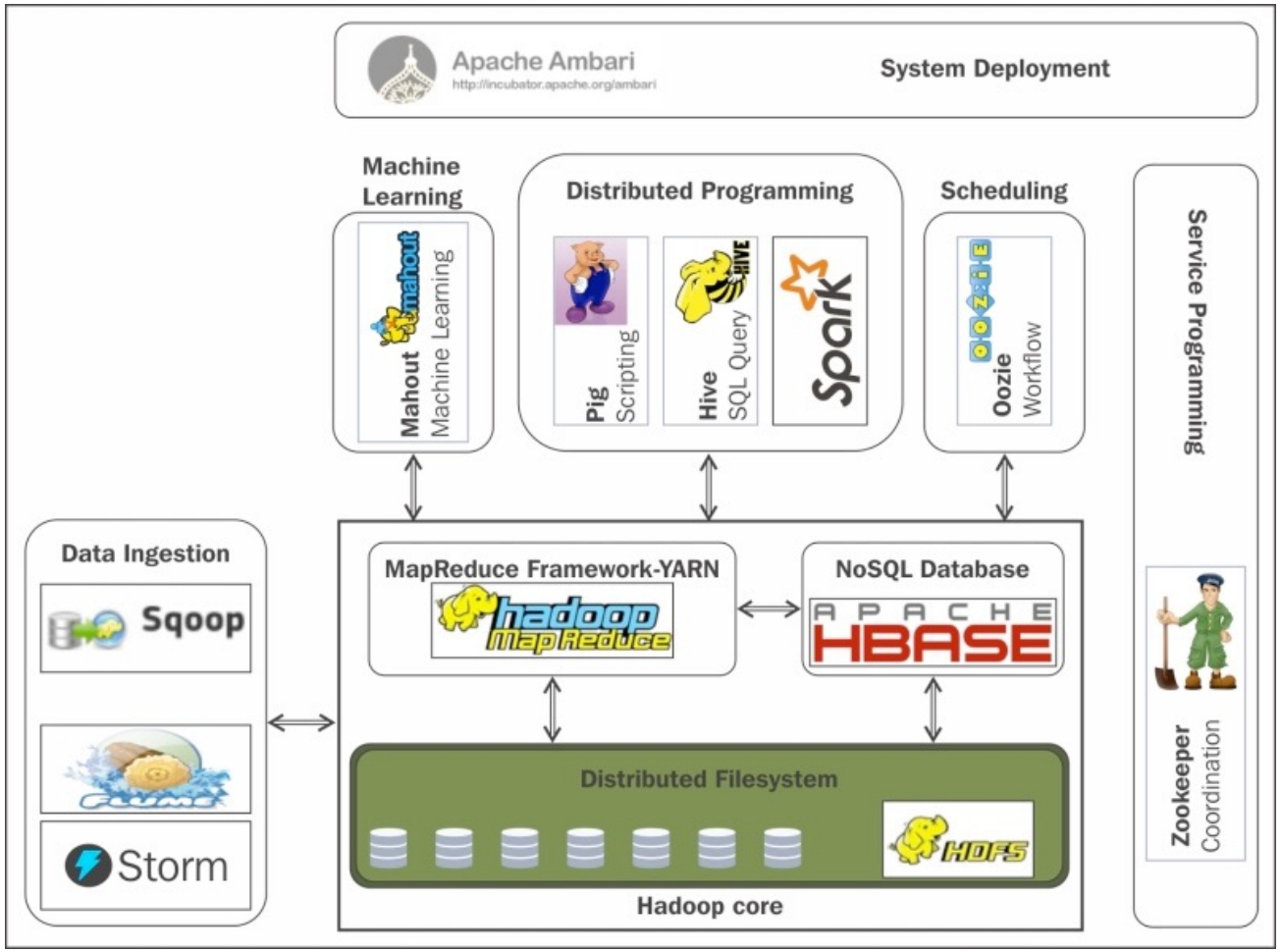


HADOOP ECOSYSTEM

CA675
Alessandra Mileo
Alessandra.mileo@dcu.ie

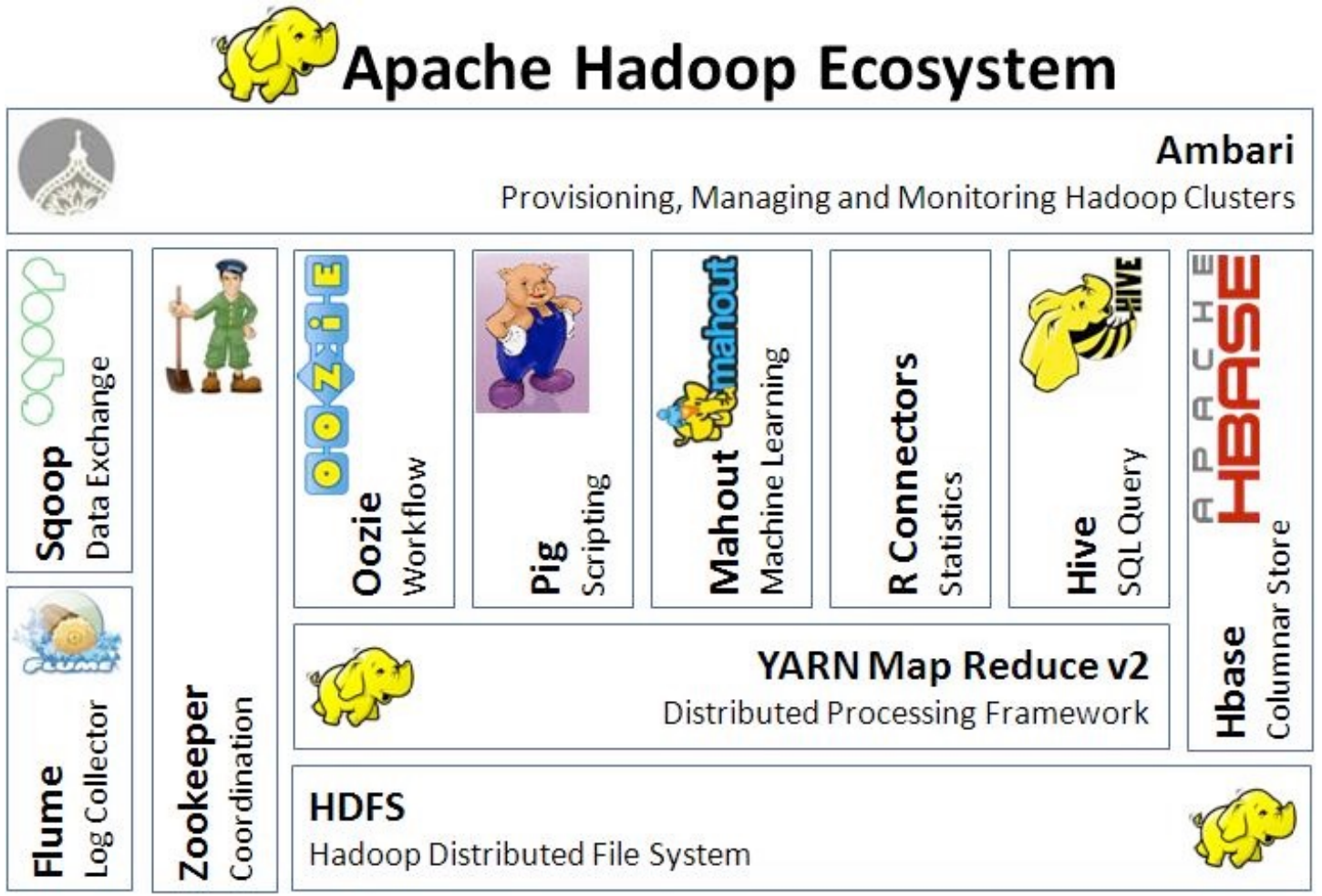
HADOOP ECOSYSTEM

- …one of many versions



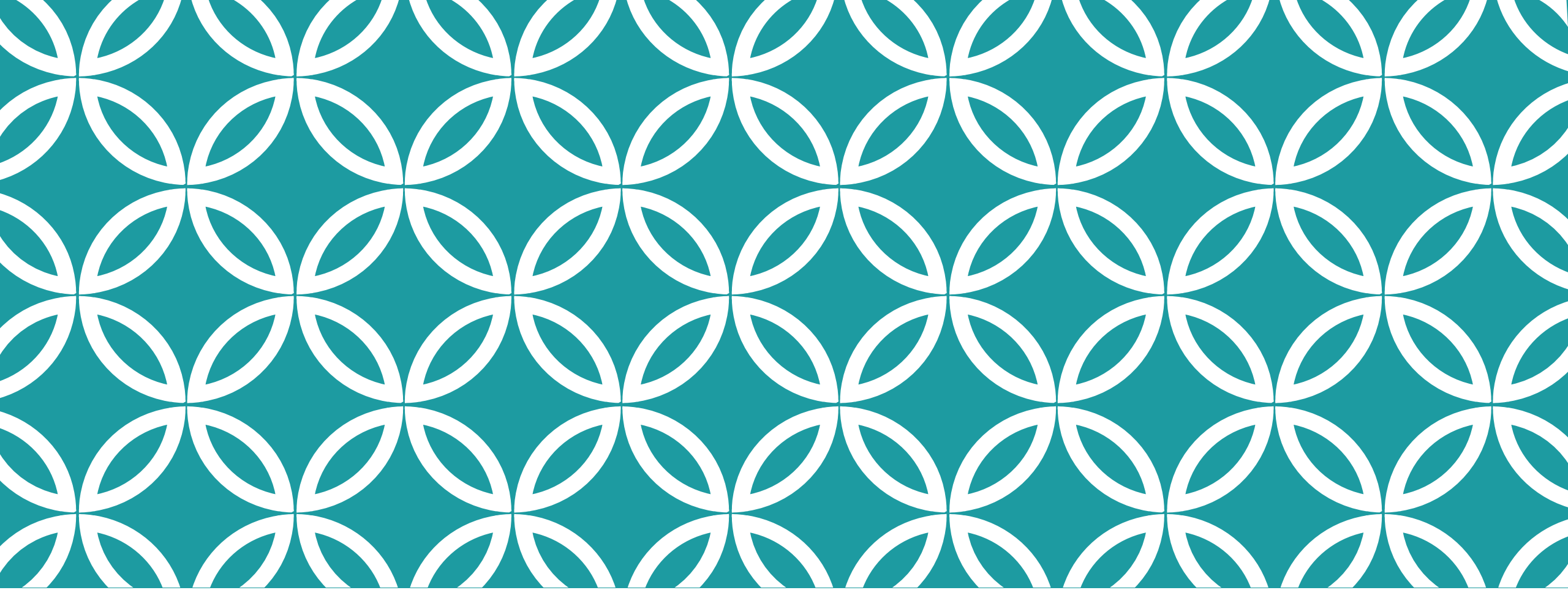
HADOOP ECOSYSTEM

- …one of many versions



OUTLINE

1. Part I
   1. Recap: MapReduce
   2. Modern Big Data Architectures
   3. Apache Hive vs Pig vs. MR
2. Part II
   1. Apache Hive
3. Part III
   1. Apache PIG



PART I

Recap: MapReduce
Modern Big Data Architectures
Apache Hive vs Pig vs. MR

MAPREDUCE FUNCTIONS

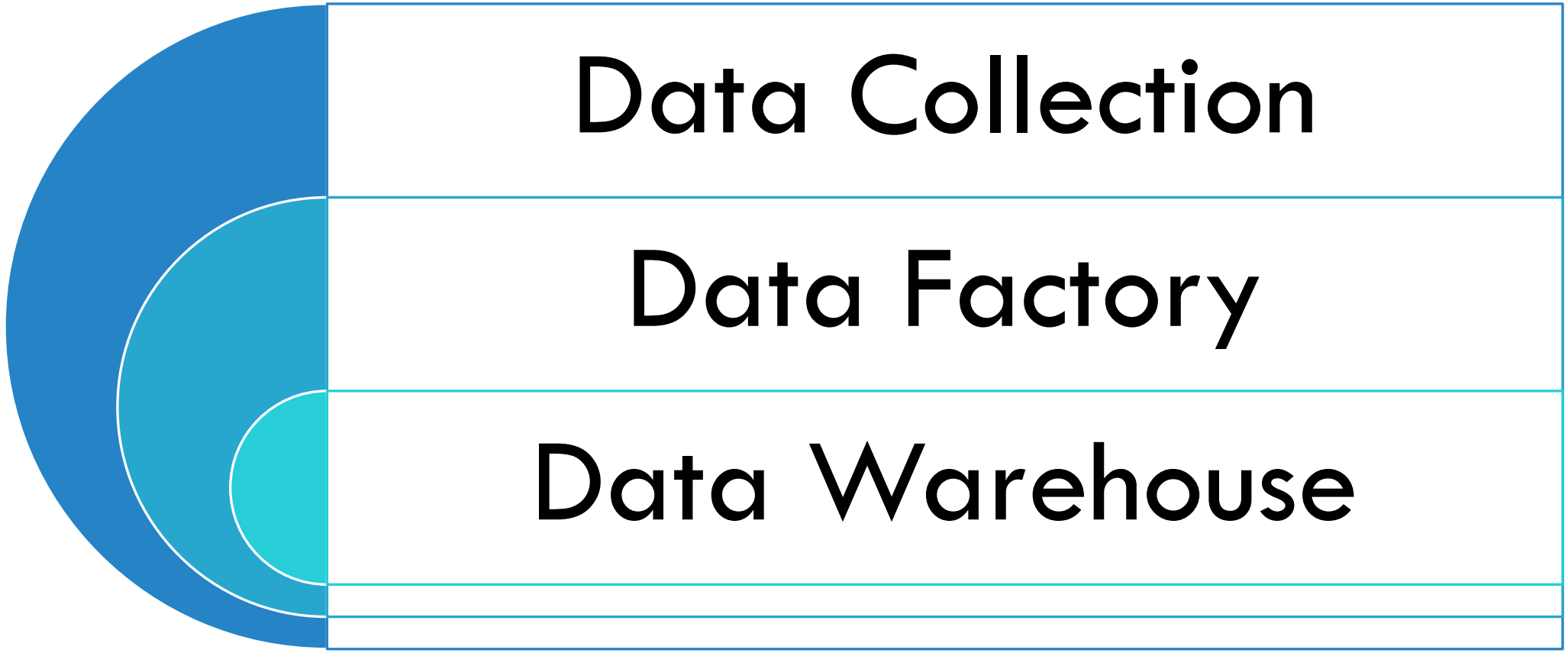
$$Map(K_{in}, V_{in}) \rightarrow list(K_{inter}, V_{inter})$$

$$Reduce(K_{inter}, list(V_{inter})) \rightarrow list(K_{out}, V_{out})$$

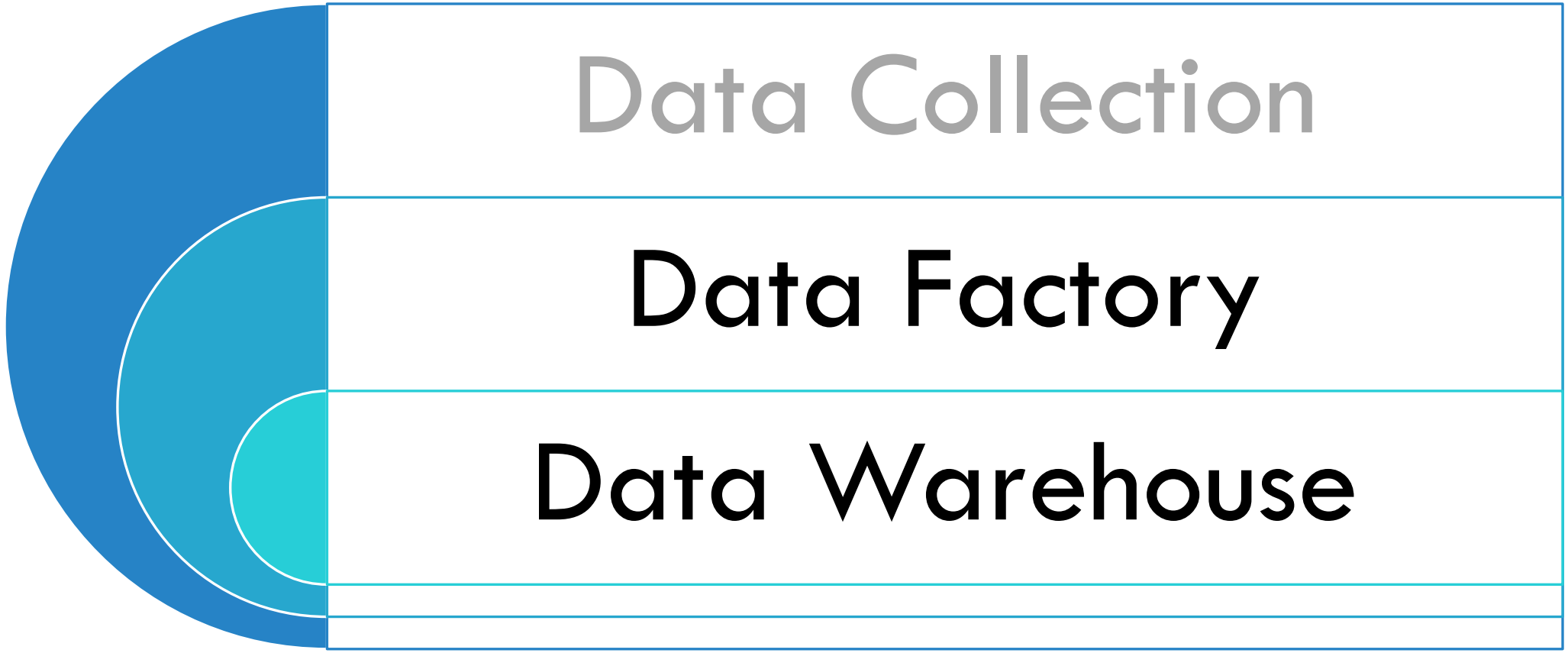
To Mapreduce or to not Mapreduce?

- Write a program every time you work with data?
- Does everyone have to be a programmer?
- What if you want to *explore* the data?
- What if you need many, relatively simple, operations?

DATA PROCESSING LAYERS



DATA PROCESSING LAYERS



DATA FACTORY

- Take Raw Data and create a refined product that can be queried
  - Extract, Transform, Load
- 'Clean' the data
- Data Pipelines
  - Uniform data assembly lines that produce refined data from raw data
- Iterative Processing
  - Rather than completely fresh data, incrementally add to existing (large amount of) data
- Research
  - Improve, evaluate and measure the ETL Process

DATA WAREHOUSE

- Store and retrieve refined data for use
- Allow for *ad-hoc* queries by Business Analysts (non-engineers)
- Integrate with Business Intelligence / Decision Software
- Allow non-programmers access

HIVE AND PIG (ON HADOOP)

Apache Hive

- SQL-Style Query Language
- Works off a structured schema
- Built around tables, rows, columns and queries
- Java/Hadoop API knowledge not required

Apache Pig

- Imperative Dataflow language
- Works on semi-structured data
- Has extensive support for transforming and cleaning data
- Is like a (scripting) programming language

HIVE AND PIG (ON HADOOP)

Apache Hive

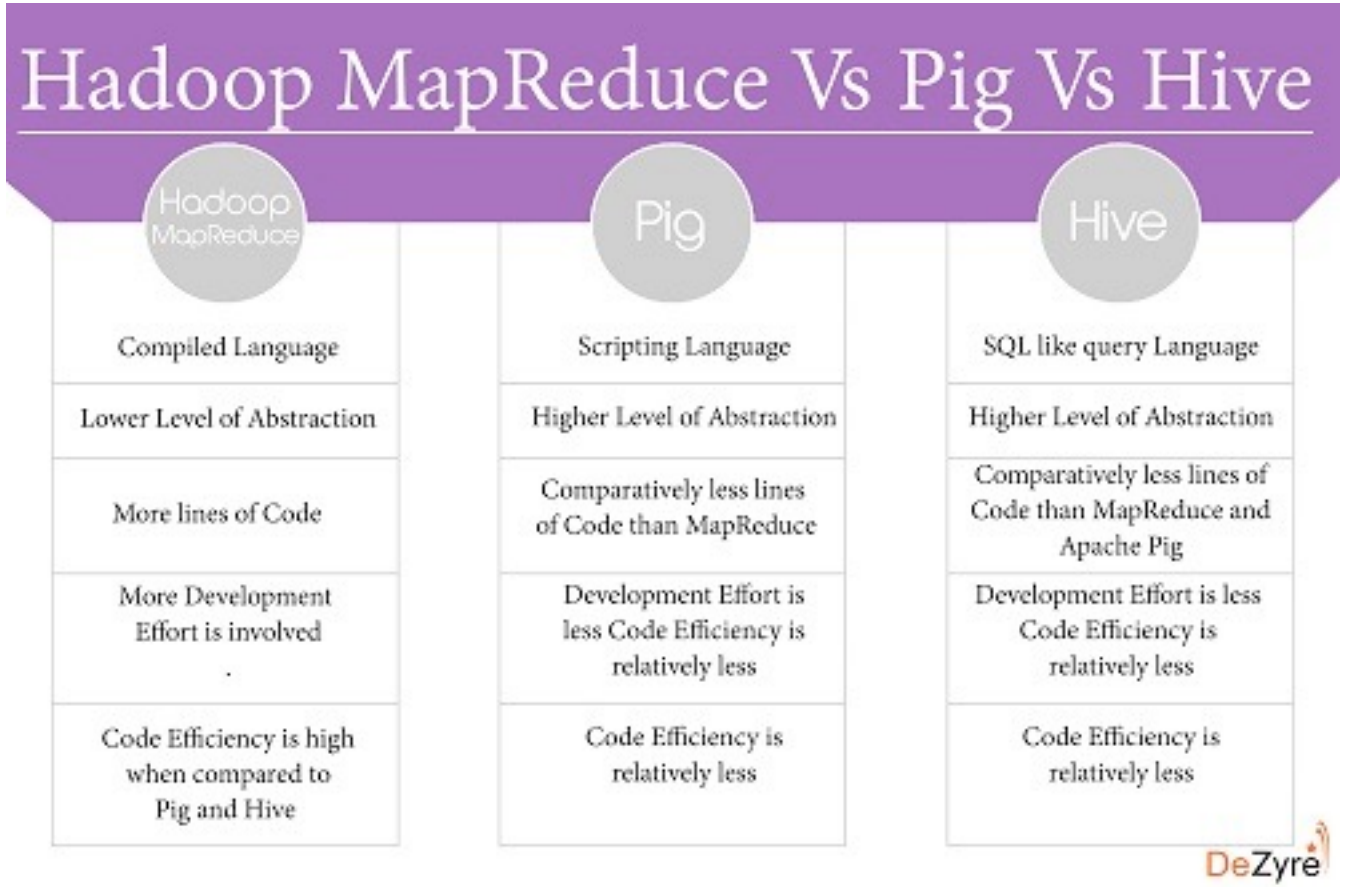
- SQL-Style Query Language
- Works off a structured schema
- Built around tables, rows, columns and queries
- Java/Hadoop API knowledge not required

Apache Pig

- Imperative Dataflow language
- Works on semi-structured data
- Has extensive support for transforming and cleaning data
- Is like a (scripting) programming language

- Not rapid-response (code efficiency is low)
- Focused on read-heavy applications

MAPREDUCE VS PIG VS HIVE



WordCount Example In MapReduce

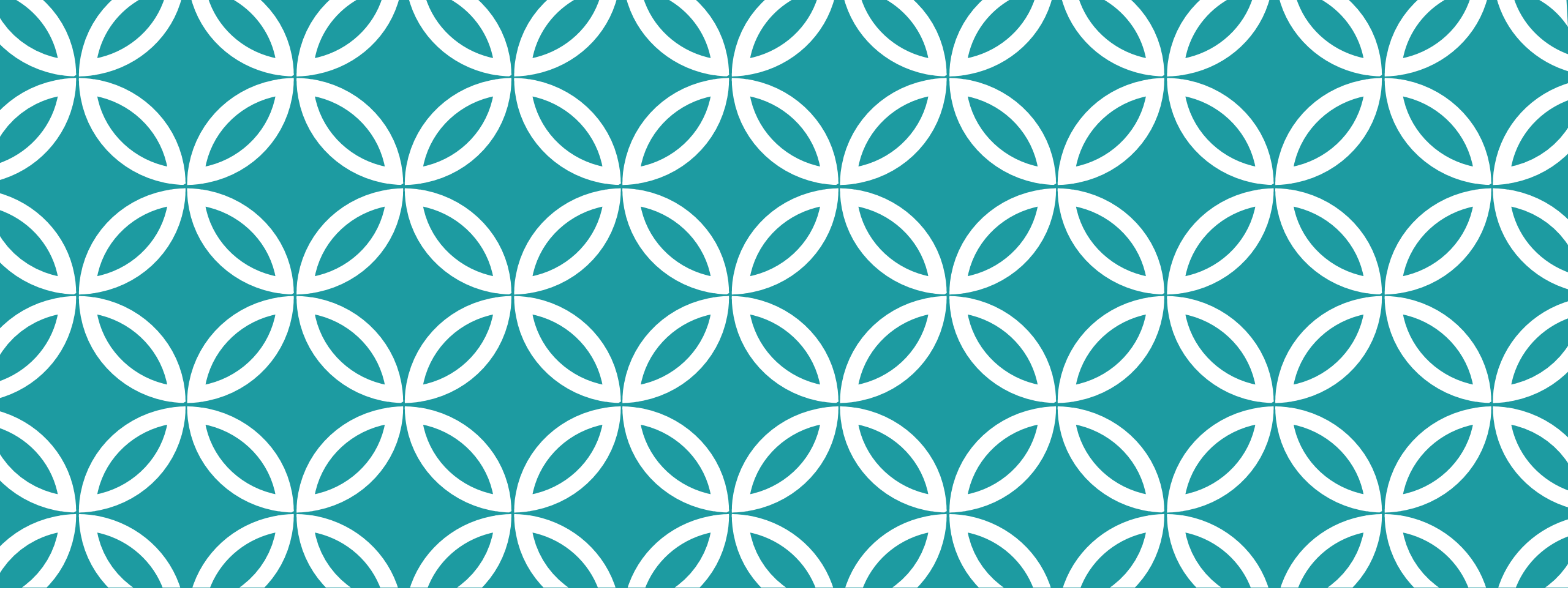
```
public class WordCount {
 public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
   private final static IntWritable one = new IntWritable(1);
   private Text word = new Text();

   public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
      String line = value.toString();
      StringTokenizer tokenizer = new StringTokenizer(line);
      while (tokenizer.hasMoreTokens()) {
         word.set(tokenizer.nextToken());
         context.write(word, one);
      }
   }
 }

 public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
   public void reduce(Text key, Iterable<IntWritable> values, Context context)
     throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
         sum += val.get();
      }
      context.write(key, new IntWritable(sum));
   }
 }

 public static void main(String[] args) throws Exception {
   Configuration conf = new Configuration();

   Job job = new Job(conf, "wordcount");
   job.setOutputKeyClass(Text.class);
   job.setOutputValueClass(IntWritable.class);

   job.setMapperClass(Map.class);
   job.setReducerClass(Reduce.class);

   job.setInputFormatClass(TextInputFormat.class);
   job.setOutputFormatClass(TextOutputFormat.class);

   FileInputFormat.addInputPath(job, new Path(args[0]));
   FileOutputFormat.setOutputPath(job, new Path(args[1]));

   job.waitForCompletion(true);
 }
}
```

WordCount Example By Pig

```
A = LOAD 'wordcount/input' USING PigStorage as (token:chararray);

B = GROUP A BY token;

C = FOREACH B GENERATE group, COUNT(A) as count;

DUMP C;
```

WordCount Example By Hive

```
CREATE TABLE wordcount (token STRING);

LOAD DATA LOCAL INPATH 'wordcount/input'
OVERWRITE INTO TABLE wordcount;


SELECT count(*) FROM wordcount GROUP BY token;
```



PART II

Apache Hive

https://www.slideshare.net/nzhang/hive-training-motivations-and-real-world-use-cases

WHAT IS HIVE

more data → more queries → more speed

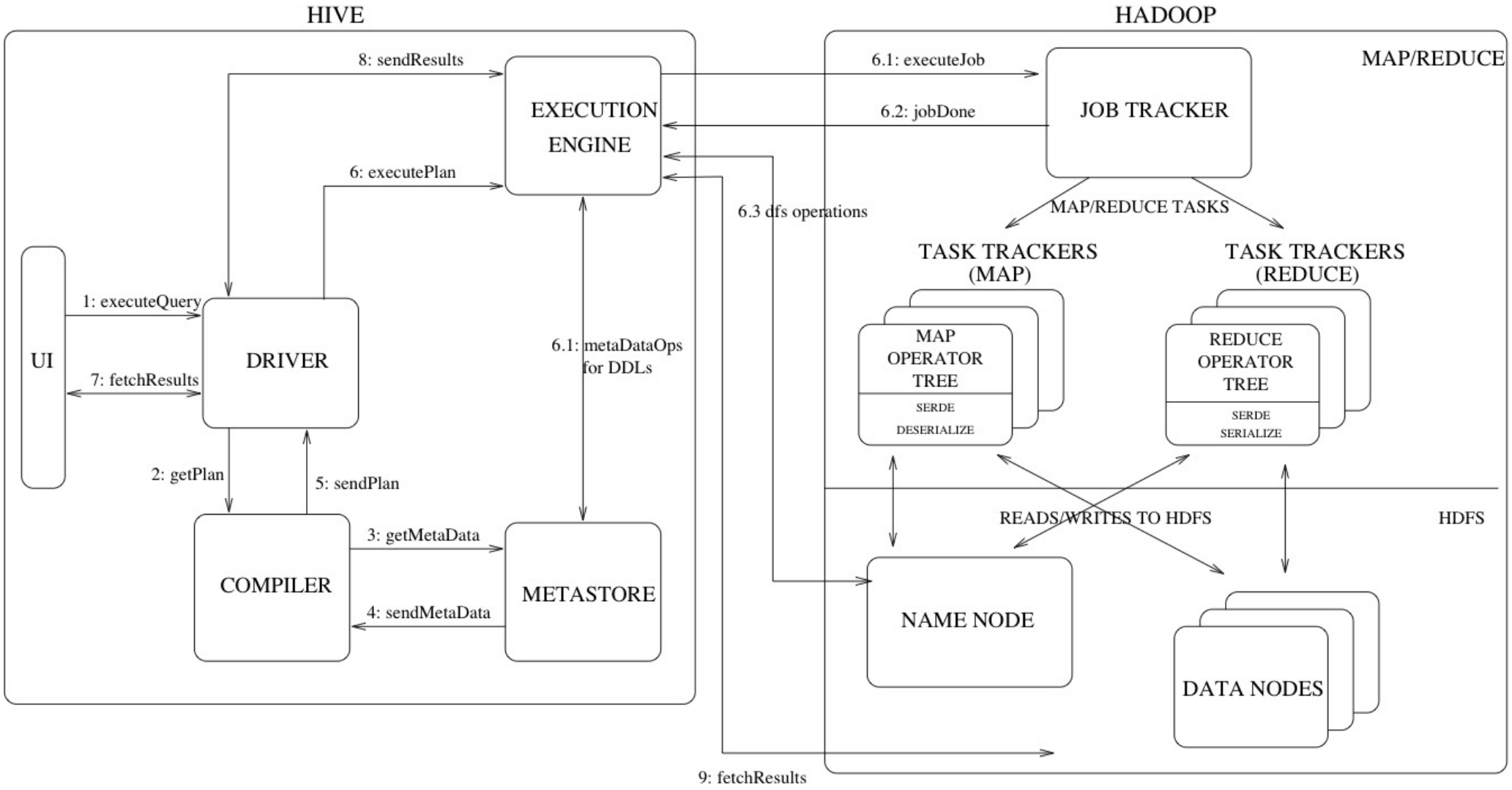
- Facebook need: too much data (exponential growth) stored in Oracle DB every night
- Efficient Data Warehouse infrastructure:
  - Built on top of Hadoop (SQL-like interface to Hadoop)
  - Can compile SQL-like queries in to MapReduce jobs and run them on Hadoop cluster

HIVE IS GOOD/NOT GOOD FOR

- Good for:
  - Integration with Business Intelligence
  - Work with structured data
  - Developers familiar with SQL
- Not Good for:
  - Real-time queries
  - Unstructured data
  - Complex User Defined Functions
  - Complex queries (e.g. many joins)
  - Whenever you need lower level control

https://cwiki.apache.org/confluence/display/Hive/Design

HIVE ARCHITECTURE



HIVE DATA DEFINITION LANGUAGE IN HIVEQL

- **Databases** Contain **Tables**
- **Tables** Contain
  - **Columns** (schema)
  - **Rows** (values)
- **Partitions** organise data based on value of one or more keys
- **Buckets/Clusters** group data for easier partitions (based on value of a hash function of some column)

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

EXAMPLE: PAGEVIEWS TABLE

Schema (columns)

Bucket (User_ID)

Values

ip	url	url_reference	User_ID	time
136.206.15.50	http://www.example.com/	http://www.google.com/	20334000	1456046590
136.206.1.4	http://www.example.com/about	http://www.example.com/	42099913	1456137790
136.206.1.4	http://www.example.com/about	http://www.example.com/	20334000	1504344300
134.226.83.67	http://www.example.com/code	http://www.example.com/test	57734109	1456143312

Partition (data from "2016-12-3”1)

HIVEQL

- SQL-like language, can embed custom scripts in any language
- Ability to filter rows from a table using a where clause.
- Ability to select certain columns from the table using a select clause.
- Ability to do equi-joins between two tables.
- Ability to evaluate aggregations on multiple "group by" columns for the data stored in a table.
- Ability to store the results of a query into another table.
- Ability to download the contents of a table to a local (e.g., nfs) directory.
- Ability to store the results of a query in a hdfs directory.
- Ability to manage tables and partitions (create, drop and alter).
- Ability to plug in custom scripts in the language of choice for custom map/reduce jobs.

https://acadgild.com/blog/hive-real-life-use-cases/

HIVE EXAMPLE: TABLE 'WWW_ACCESS'

ip	url	time
136.206.15.50	http://www.example.com/	1456046590
136.206.1.4	http://www.example.com/about	1456137790
134.226.83.67	http://www.example.com/test	1456143312

HOW MANY ROWS?

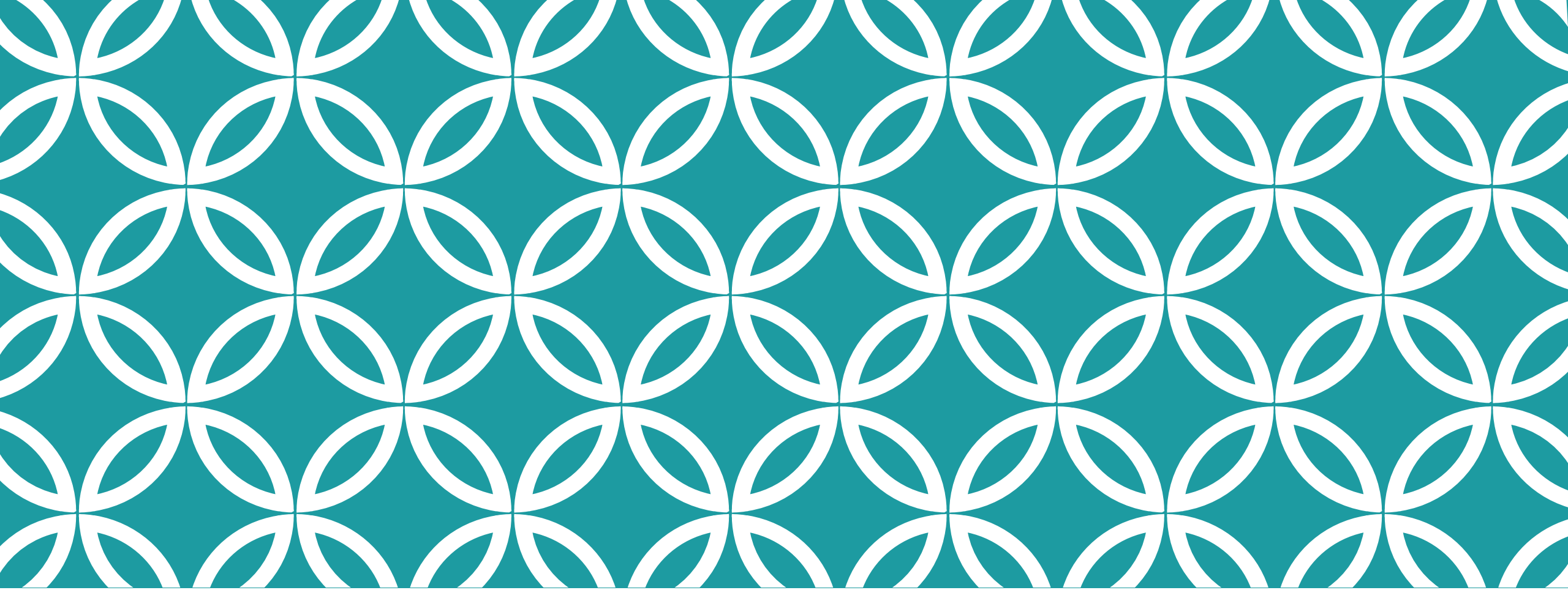
```
SELECT COUNT(*) FROM www_access;
```

HOW MANY DIFFERENT IPS VISITED THE ROOT?

```
SELECT COUNT(distinct ip) FROM www_access  \
    WHERE url=‘http://example.com/';
```

TOP 10 IPS (BY NUMBER OF ACCESSES)

```
SELECT ip, COUNT(*) AS cnt FROM www_access \
    GROUP BY ip
    ORDER BY cnt DESC LIMIT 10;
```



PART III

Apache PIG

MAP REDUCE IS AWESOME

A way to handle Massive Data sets in a

- Robust,
- Scalable
- Fault-tolerant,
- Resource-elastic

Almost don't care how big the data is

Writing map and reduce jobs is easier than manually managing scaling

BUT…

Writing code is hard

Real-world data sets rarely arrive in an orderly form

- Missing values
- Need to categorise, bucket or group data
- Clean out certain content (e.g. stripping html)

Writing custom scripts to handle data processing problems is laborious

- Especially using MapReduce model
- Lots of map and reduce steps, with little difference

HISTORY OF APACHE PIG

Started as a research project in Yahoo! (2007)

Something high level, not quite as high as SQL, not low-level like native MapReduce

Data-flow language

Phylosophy:

1. Pigs eat anything
2. Pigs live anywhere
3. Pigs are domestic animals
4. Pigs fly

EXTRACT, TRANSFORM, LOAD

Extract

- From one or more sources
- Files, databases, sockets, streams

Transform

- Create categories, strip mark-up, round #s
- Handle errors, missing or spurious values

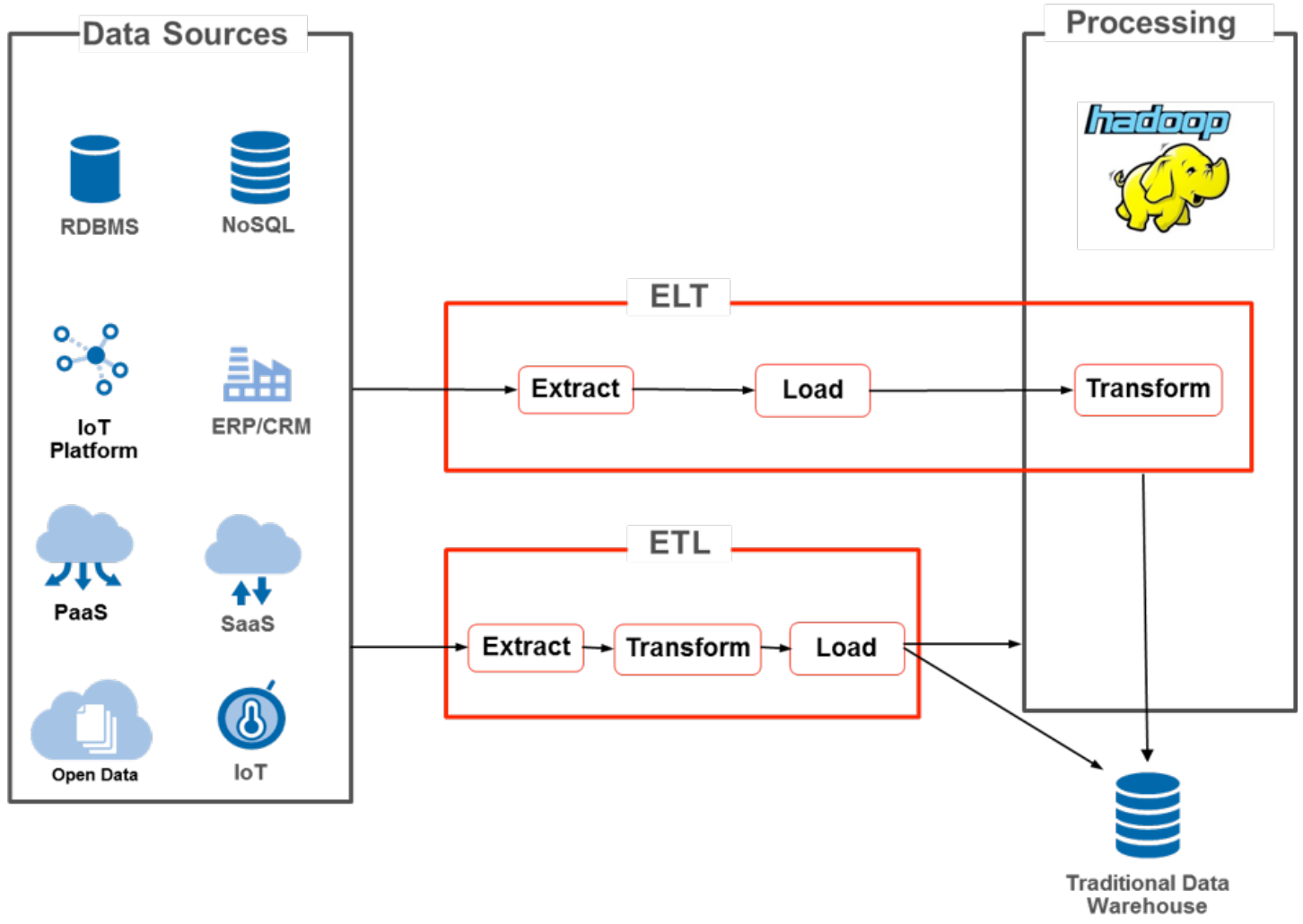
Load

- Into a form for further processing
- For example onto a Mapreduce cluster

ETL OR ELT?

Where does T happen?

When does T happen?



ETL VS ELT

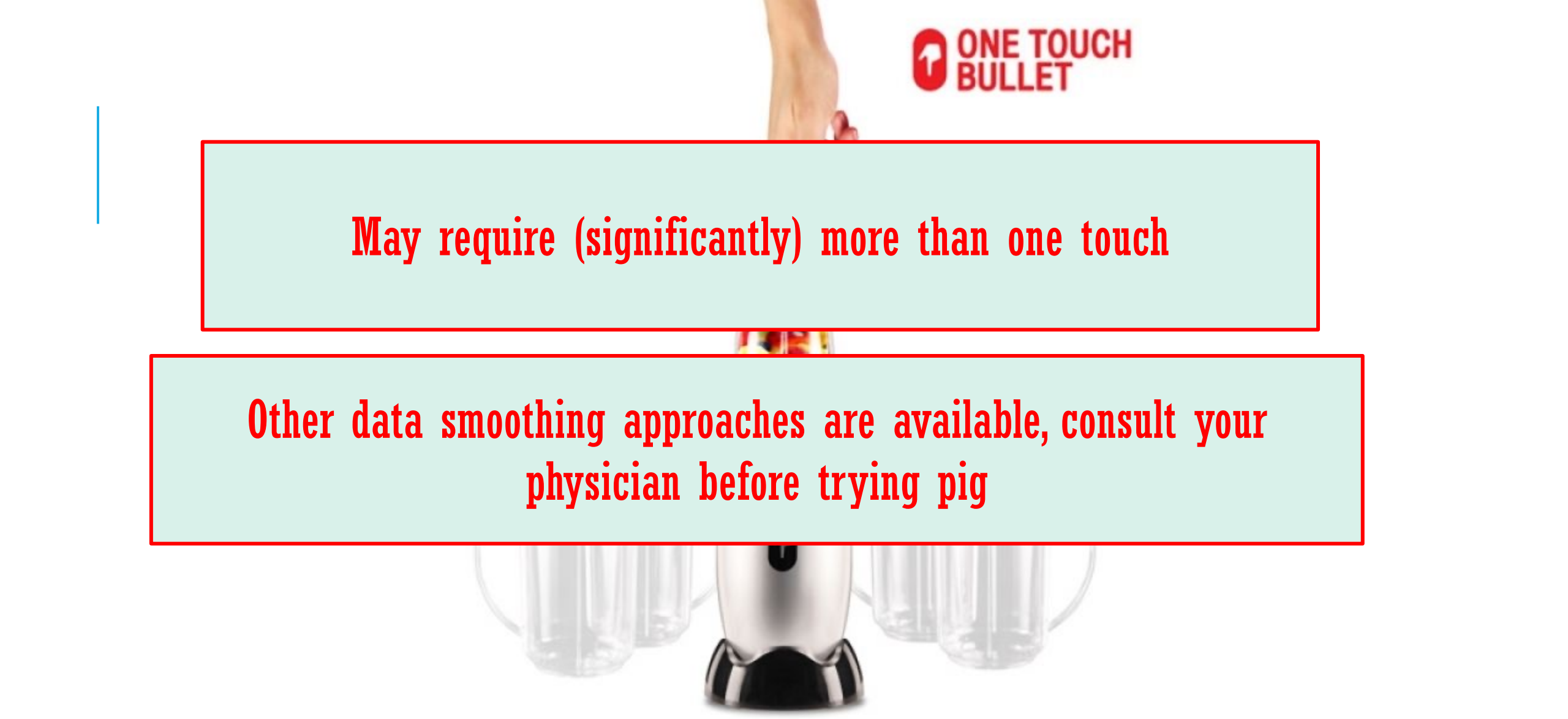
ETL: schema on write

- Data consumed as highly structured reports
- Poor quality data requiring a lot of transformation (should not be done into Hadoop)
- Can be fed into a Data Warehouse after the ETL tool
- Supported by traditional ETL tools and stored in a Data Warehouse

ELT: schema on read

- Data used by skilled data analysits, leveraging coding abilities (e.g. machine learning)
- Limited data transformation
- Can only be fed into a Data Warehouse from Hadoop after Transformation
- Supported by Hadoop/Hive & Spark and stored as Datalake


ONE TOUCH
BULLET



May require (significantly) more than one touch

Other data smoothing approaches are available, consult your physician before trying pig

http://www.datasciencecentral.com/profiles/blogs/ten-top-languages-for-crunching-big-data

PIG IN A NUTSHELL

Scripting platform for analyzing large data sets

Write complex MapReduce transformation in a scripting language

Pig translates these into MapReduce jobs so they can be executed on the Hadoop infrastructure (Yarn+HDFS)

Properties:

- Extensible (with custom functions)
- Easy to program (many small steps as data flow sequences)
- Self-optimizing (focus on semantics rather than efficiency)

RECAP: PIG IS GOOD FOR…

Many small tasks

Developers (esp. scripting)

ETL/ELT data pipelines (structured data is not a requirement)

Iterative processing (procedural-type processing)

- Batch processing

Research on raw data

PIG IS NOT GOOD FOR…

Few complex tasks (too many steps)

Complex Join operations (imagine to do them at the query plan level)

Operations relying on complex schema (debugging the schema might be a nightmare since there is a lot of flexibility)

MORE ON PIG VS HIVE/RDBS

Pig might look like SQL but…

It's a data-flow language:

- Step-by-step set of operations
- Each operation is a single transformation
- Optimization (like `Group` or `Filter`) need to be checked by hand (are they useful?)

As opposed to declarative:

- Set of constraints
- Applied to an input together to obtain an output
- Optimization often managed via transformation (e.g. query plan)

Schema is optional and can be defined as you go, not predefined and strict like in RDBS

THE CSV (COMMA-SEPARATED VALUE) FILE

movieId,title,genres

1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy

2,Jumanji (1995),Adventure|Children|Fantasy

3,Grumpier Old Men (1995),Comedy|Romance

4,Waiting to Exhale (1995),Comedy|Drama|Romance

5,Father of the Bride Part II (1995),Comedy

6,Heat (1995),Action|Crime|Thriller

7,Sabrina (1995),Comedy|Romance

8,Tom and Huck (1995),Adventure|Children

9,Sudden Death (1995),Action

First row column names (optional)

Each row is an entry

Values have comma between them